

Game Design Document: Project "Persona-33"

Dokumen Status: Draft v1.0 (Living Document) **Engine:** Unity **Genre:** Turn-Based JRPG / Life Simulation **Target Platform:** PC

1. Game Overview

Elevator Pitch: Seorang siswa SMA swasta menjalani kehidupan ganda: di siang hari ia terikat pada rutinitas sekolah, namun di malam hari ia mengeksplorasi *dungeon* misterius dengan kekuatan supernatural. Pertarungan menuntut strategi elemen dan refleks yang tajam, sementara setiap keputusan harian memiliki dampak permanen.

Scope Proyek Saat Ini (Vertical Slice): Prolog permainan yang berdurasi **7 Hari In-Game**. Pemain akan merasakan siklus sekolah penuh, tutorial sistem elemen, tutorial mekanik *active parry/critical*, dan diakhiri dengan pertarungan bos pertama.

2. Design Pillars

(Aturan utama yang menjadi fondasi pengambilan keputusan desain).

- Duality of Gameplay:** Manajemen waktu di dunia nyata (sekolah) berbanding lurus dengan kekuatan dan efisiensi pemain saat mengeksplorasi *dungeon*.
- Active Turn-Based Combat:** Pertarungan bersifat *turn-based*, namun pemain dilarang pasif. Refleks menekan tombol (*Parry/Critical*) di waktu yang sangat sempit sama pentingnya dengan eksploitasi elemen.
- Strict Separation of Logic and Presentation:** Antarmuka (UI) harus statis secara tata letak. Modifikasi logika pada sistem pembacaan data (seperti *parsing* data HP atau Status) sama sekali tidak boleh mengubah, merusak, atau menggeser UI *layout* yang sudah ada.

3. Core Gameplay Loop

Siklus harian dibagi menjadi dua fase utama yang saling mengunci:

- Fase Siang (Life Sim):** Pemain berada di sekolah. Fokus pada dialog, menjalin relasi sosial, dan menaikkan status dasar (misal: Keberanian, Pengetahuan). Sistem dijalankan melalui interaksi *Visual Novel*.
- Fase Malam (Dungeon RPG):** Pemain masuk ke *dungeon*. Fokus pada eksplorasi dan pertarungan *turn-based*. Manajemen *Health Point* (HP) dan *Skill Point* (SP) sangat krusial.

4. Mekanik Pertarungan (Turn-Based & Action)

Pertarungan dikelola menggunakan *Finite State Machine* (FSM). Setiap giliran terbagi menjadi fase-fase terisolasi:

A. Sistem Active Critical (Fase Pemain Menyerang)

- **Deskripsi:** Saat pemain memilih perintah "Attack", animasi serangan dieksekusi.
- **Mekanik:** Pada durasi *frame* tertentu saat animasi berjalan (misal: detik ke-1.2), sistem akan memunculkan *prompt* visual singkat. Jika pemain memberikan *input* tepat pada jendela waktu ini (*Critical Window*), *damage* yang dihasilkan akan digandakan.
- **Implementasi:** Menggunakan *Animation Events* di Unity untuk memicu pembacaan *input* secara *real-time*.

B. Sistem Active Parry (Fase Musuh Menyerang)

- **Deskripsi:** Saat giliran AI musuh mengeksekusi serangan.
- **Mekanik:** Sebelum proyektil atau pukulan mengenai karakter pemain, terdapat *Parry Window* yang sangat sempit. Jika *input* pertahanan ditekan tepat waktu, *damage* musuh dinegasikan (menjadi 0) dan memicu animasi menangkis.

C. Elemental Weakness & "One More" System

- **Sistem Elemen:** Setiap entitas (Pemain dan Musuh) memiliki elemen dasar (Fisik, Api, Es, dll) dan kelemahan yang spesifik.
- **Mekanik:** Mengeksploitasi kelemahan target menggunakan *Skill* yang sesuai akan memberikan status "Down" pada target. Karakter yang berhasil mengeksploitasi kelemahan tersebut akan langsung mendapatkan satu giliran tambahan (Sistem *One More*).

D. Sistem "Heat Gauge" & Extreme QTE (Quick Time Events)

Deskripsi: Selain HP dan SP, terdapat sebuah parameter fluktuatif bernama **Heat Gauge** yang dimiliki oleh tim pemain maupun musuh tingkat tinggi (*Boss/Elite*). Saat *Heat Gauge* penuh, entitas tersebut akan memicu "Heat Mode", merubah fase pertarungan menjadi sekuens aksi instan (*Quick Time Event*).

Kondisi Pemicu (Trigger):

- **Akumulasi Damage:** *Heat* akan bertambah seiring jumlah akumulasi *damage* yang diberikan dan diterima.
- **Eksplorasi Elemen:** Mengeksploitasi kelemahan target secara berurutan (memicu status *Down*) akan memberikan bonus pengisian *Heat* dalam jumlah besar.

Mekanisme Heat Mode - Pemain (Offensive QTE):

- Saat *Heat Gauge* karakter/tim penuh, pemain dapat mengaktifkan *Heat Mode* sebagai aksi *free-turn*.
- Sistem *Turn-Based* akan dijeda sementara (*Time Freeze*). Kamera berubah menjadi dinamis (*cinematic angle*).
- **Extreme QTE:** Pemain harus menekan tombol aksi yang muncul di layar dengan sangat cepat (*button mashing*) atau dalam urutan ritmis tertentu yang disinkronisasikan dengan tempo ketukan musik EDM (*rhythm sequence*).
- Semakin akurat dan cepat *input* QTE yang dilakukan, semakin besar *multiplier damage* atau durasi *buff* yang dihasilkan di akhir sekuens.

Mekanisme Heat Mode - Musuh (Defensive QTE):

- Saat *Heat Gauge* musuh penuh, musuh memasuki fase *Rage* dan bersiap meluncurkan *Ultimate Attack*.
- Alih-alih sekadar menerima *damage*, pertarungan berubah menjadi mode bertahan interaktif bagi pemain.
- **Survival QTE:** Pemain dihadapkan pada sekuens *Parry* atau *Dodge* beruntun dalam waktu sepersekian detik. Gagal menekan *input* tepat waktu pada QTE ini akan berakibat pada *party-wipe* (kalah instan) atau menerima kerusakan masif.

Implementasi Teknis & Aturan Arsitektur:

- **State Machine Tambahan:** Tambahkan state baru pada *Finite State Machine* (FSM), yaitu *QteSequenceState*. Saat *state* ini aktif, siklus *Turn-Based* dihentikan murni untuk mendengarkan pendeteksian *input* kecepatan tinggi.
- **Event-Driven UI (Kepatuhan Arsitektur):** Sama seperti aturan dasar proyek, elemen visual QTE (tombol yang muncul di layar) harus dirancang secara statis di dalam *Canvas*. Sistem yang membaca *input* QTE di latar belakang hanya memanggil *Event* untuk menghidupkan/mematikan visibilitas (*enable/disable*) *GameObject* tombol tersebut, **tanpa** menggeser tata letak *RectTransform* atau memodifikasi hirarki UI yang sudah ada.
-

5. UI & Technical Architecture

Karena pengembangan game ini sangat bergantung pada integrasi data dan logika status, arsitektur teknis disusun dengan aturan ketat:

A. Data-Driven Design (Scriptable Objects)

Seluruh statistik tidak boleh di-*hardcode* di dalam *script*.

- **Enemy Data:** Menyimpan HP maksimal, Elemen, Kelemahan, dan daftar *Skill* AI.
- **Player Data:** Menyimpan *Level*, *Base Stats*, dan *Skill List*. Pembuatan musuh baru atau penyesuaian keseimbangan game (*balancing*) dilakukan murni melalui *Inspector* di Unity.

B. Aturan Ketat Pengembangan UI & Parsing Data

Untuk mencegah kerusakan desain visual saat sistem berkembang, proyek ini menerapkan pemisahan antara pengolahan data dan presentasi visual:

- **Data Parsing:** *Script* yang bertugas mengolah, menghitung *damage*, atau membaca *Scriptable Objects* hanya bekerja di balik layar.
- **UI Layout:** Segala bentuk *layouting* (seperti bar HP, menu *turn-based*, indikator *Parry*) dibuat statis. Ketika *programmer* atau sistem melakukan pembaruan kode untuk *parsing* data, jangan pernah memodifikasi *layout* UI yang sudah ditetapkan.
- **Komunikasi (Event Driven):** Penghubung antara data dan UI menggunakan *Event System* (*UnityEvent* atau C# *Action*). Contoh: Saat pemain terkena *damage*, *Controller* menyiarkan *event* pembaruan, dan UI hanya menangkap sinyal tersebut untuk memanipulasi *fill amount* pada bar HP, tanpa menyentuh *RectTransform* atau struktur *layout*.

6. Jadwal Pengembangan Prolog (Milestones)

1. **Minggu 1:** *Setup* repositori, pembuatan *State Machine* dasar untuk *Turn-Based* (tanpa UI, berbasis *Console Log*).
2. **Minggu 2:** Pembuatan arsitektur data (*Scriptable Objects* untuk Musuh, Karakter, dan *Skill*).
3. **Minggu 3:** Implementasi sistem mekanik *Active Critical* dan *Active Parry* menggunakan kotak/objek 3D standar (sebagai prototipe animasi).
4. **Minggu 4:** Pembuatan sistem Dialog statis dan manajemen Kalender Dasar (Hari 1-7).
5. **Minggu 5:** Integrasi UI statis (Menu pertarungan, indikator status) dengan sistem data (implementasi *Event Driven UI*).

7. Art & Audio Direction

A. Audio & Music Style (EDM Focus)

Pendekatan musik dalam proyek ini berpusat pada genre **EDM (Electronic Dance Music)** untuk membangun identitas modern, *stylish*, dan *energetic*. Musik dirancang dinamis menyesuaikan dua fase utama permainan:

- **Fase Siang (Life Sim & School):** Menggunakan sub-genre *Chillwave*, *Lo-Fi Electronic*, atau *Ambient EDM*. Musik memiliki ketukan yang santai (sekitar 90-100 BPM) dengan dominasi suara *synthesizer* yang lembut. Tujuannya untuk memberikan rasa tenang saat pemain merencanakan aktivitas harian, membaca dialog, dan mengatur strategi.
- **Fase Malam (Dungeon Exploration):** Menggunakan tempo medium (110-120 BPM) seperti *Deep House* atau *Trance* dengan atmosfer yang lebih gelap dan misterius untuk membangun ketegangan selama penjelajahan.
- **Fase Pertarungan (Active Combat):** Transisi mulus ke sub-genre *Up-tempo EDM* seperti *Electro House*, *Drum and Bass*, atau *Synthwave* bertenaga tinggi (130+ BPM). *Bassline* yang tebal dan ketukan yang menghentak (*punchy kick*) berfungsi

untuk memompa adrenalin pemain agar fokus pada mekanik *Active Parry* dan *Active Critical*.

- **Efek Suara (SFX):** SFX pertarungan (suara tebasan, tembakan sihir, penanda kelemahan) menggunakan elemen suara digital/glitch yang diselaraskan (*sync*) dengan ketukan musik EDM, sehingga pertarungan terasa ritmis. SFX UI di menu juga menggunakan bunyi-bunyi elektronik modern yang bersih dan responsif.

8. Resource & Tooling Pipeline (100% Free / Open-Source)

Karena fokus pengembangan saat ini adalah membangun *Vertical Slice* dan menguji arsitektur *codebase*, seluruh aset visual dan audio akan menggunakan *placeholder* atau aset gratis bersertifikat.

A. Sumber Aset Visual (3D, 2D, & Animasi)

- **Animasi Karakter (Krusial untuk Mekanik Parry/Critical):** Menggunakan **Mixamo** (gratis dengan akun Adobe). Mixamo menyediakan ribuan animasi *idle*, *attack*, dan *hit reaction* yang bisa langsung diunduh dan dipasangkan *Animation Event* di Unity untuk mengatur *timing* sistem kombat aksi.
- **Model 3D Karakter & Lingkungan:**
 - **Kenney.nl:** Sumber utama untuk model 3D *low-poly* gratis (CC0). Sangat ringan dan cocok untuk membangun prototipe *dungeon* dengan cepat.
 - **Sketchfab:** Mencari aset dengan filter lisensi "CC0" (bebas hak cipta) untuk properti spesifik.
- **Aset UI (User Interface):** Menggunakan koleksi UI Pack dari **Kenney.nl**.
 - *Catatan Penting Integrasi UI:* Saat mengimpor *sprite* tombol atau panel dari Kenney, aset ini hanya digunakan untuk mengganti komponen **Source Image** pada objek UI Unity. Pemasangan aset baru ini sama sekali tidak boleh mengubah **RectTransform**, merusak, atau menggeser tata letak UI yang sudah dibangun secara statis sebelumnya.

B. Sumber Audio (EDM & SFX)

- **Musik EDM (BGM):**
 - **OpenGameArt.org:** Mencari *track* bergenre *Trance*, *Synthwave*, atau *Electro House* dengan filter pencarian lisensi CC0 atau CC-BY.
 - **YouTube Audio Library / Incompetech:** Sumber musik *royalty-free* untuk *ambient* sekolah (Fase Siang).
- **Efek Suara (SFX):**
 - **Freesound.org:** Digunakan untuk mencari suara spesifik seperti bunyi *glitch*, *digital beep* untuk UI, dan efek benturan (*hit*) untuk mekanik *Parry/Critical*.
- **Modifikasi Audio:** Jika perlu memotong lagu EDM atau memberikan efek *fade-in/out*, gunakan perangkat lunak *open-source* seperti **Audacity** atau **LMMS** (Linux MultiMedia Studio, sangat cocok untuk mengutak-atik *track* elektronik).

C. Development Tools (Perangkat Lunak)

- **Game Engine:** Unity (Personal Edition).
- **Code Editor:** Visual Studio Code (ringan, integrasi C# yang cepat, dan sangat andal di lingkungan pengembangan berbasis Linux/Ubuntu maupun Windows).
- **3D Adjustment: Blender** (Gunakan hanya untuk sedikit memperbaiki skala model 3D atau rigging ringan dari Mixamo sebelum dimasukkan ke Unity, bukan untuk *modeling* dari awal).

9. Version Control & Manajemen Proyek

Mengingat kompleksitas arsitektur data (*State Machine* dan *Scriptable Objects*) yang akan dibangun, proyek ini diwajibkan menggunakan *Version Control System*.

- **Git & GitHub / GitLab:** Setiap fitur baru (misal: *Sistem Kalender*, *Sistem Critical*) dikerjakan di *branch* terpisah. Jangan pernah menyatukan (*merge*) kode ke *main branch* sebelum fitur tersebut teruji tidak memecahkan siklus *Turn-Based Combat*. File `.gitignore` standar Unity wajib diaktifkan agar repositori tidak membengkak.
- **Dokumentasi Task:** Menggunakan Trello atau GitHub Projects (Kanban Board) untuk melacak *task* harian, terutama saat mengatur sinkronisasi antara *Animation Event* dan kalkulasi *damage* di *script C#*.

10. Save/Load Architecture & Data Serialization

Game JRPG memiliki banyak sekali *state* yang harus disimpan (Hari ke-berapa, status sosial, level karakter, *inventory*, dan kemajuan *dungeon*). Di industri, arsitektur penyimpanan ini harus didefinisikan sejak awal agar data tidak korup.

- **Format Data:** Menggunakan format **JSON** untuk serialisasi data penyimpanan (*Save Data*). Unity memiliki utilitas `JsonUtility` bawaan yang sangat cepat, atau bisa menggunakan *library* pihak ketiga seperti `Newtonsoft.Json` untuk struktur data yang lebih kompleks.
- **Pemisahan Data Pemain vs Data Master:** File *save* hanya boleh menyimpan ID referensi (misal: `WeaponID: 001`), bukan seluruh statistik senjata tersebut. Statistik asli tetap dibaca dari *Scriptable Objects*.
- **Aturan Ketat Parsing Data ke UI (Krusial):** Saat pemain memuat (*load*) data dari JSON kembali ke dalam game (misalnya memuat status HP atau daftar *item* ke layar menu), proses *parsing* data ini **sama sekali tidak boleh mengubah, menggeser, atau merusak *layout* UI yang sudah ada**. *Script* hanya bertugas membaca nilai JSON dan mengganti teks atau gambar (*sprite*), sementara struktur *RectTransform* UI harus tetap statis dan terkunci rapat.

11. Text Management & Localization Pipeline

Game *Persona* dan JRPG pada umumnya adalah game yang 80% isinya adalah membaca teks. Menulis kalimat (*hardcode*) langsung di dalam *script* C# adalah larangan besar di industri.

- **External String Database:** Semua teks (Nama *Item*, Deskripsi *Skill*, Dialog, Menu UI) harus disimpan di luar *script* utama, menggunakan format CSV atau JSON.
- **Sistem Pembacaan (Parsing):** Buat sebuah *Localization Manager* (atau gunakan Yarn Spinner) yang mengambil teks berdasarkan *String_ID* (misal: *DIA_PROLOGUE_01*). Ini memastikan jika di kemudian hari Anda ingin mengubah struktur kalimat atau menerjemahkan game ke bahasa lain, Anda tidak perlu menyentuh satu baris pun kode pemrograman utama atau membongkar *layout* UI.

12. Build Pipeline & CI/CD (Continuous Integration/Continuous Deployment)

Di studio profesional, proses *compile* game menjadi aplikasi yang bisa dimainkan (.exe atau .apk) tidak dilakukan secara manual satu per satu oleh *programmer*, melainkan diotomatisasi.

- **Automated Builds:** Menggunakan **GitHub Actions** untuk membuat *pipeline* CI/CD. Setiap kali Anda melakukan *Push* atau *Merge* kode utama (terutama yang menyangkut perubahan pada *State Machine* kombatan), GitHub Actions akan secara otomatis menjalankan proses *build* proyek Unity di *cloud*.
- **Environment Standar:** Pastikan *pipeline* dan lingkungan pengembangan mendukung *cross-platform*, mengingat pengembangan dapat dilakukan di berbagai sistem operasi, termasuk lingkungan berbasis **Ubuntu Linux** (Unity Editor memiliki dukungan *native* untuk Linux, sehingga proses *scripting* dan *version control* akan berjalan mulus).
- **Testing:** *Build* otomatis ini berfungsi sebagai *sanity check* untuk memastikan perubahan sistem atau pembaruan UI *parsing* tidak merusak *file* eksekusi (*executable*) utama sebelum dimainkan.